



Applications of Hierarchical Density-Based Spatial Clustering of Applications with Noise (HDBSCAN) in Computer Vision Analysis

GR 5205 Final Project - Spring 2025

Jason Ye, Nikhil Shanbhag, Kei Ogawa, Daniel Shi



The Problem

- **Goal:** Estimate object dimensions in blurred images using unsupervised clustering, and more specifically, HDBSCAN
- **Broader application:** Medical imaging, object recognition, quality control
- **Motivation:** Traditional methods struggle with noise and low contrast, so agglomerative clustering offers a generalizable usage in real world applications

Background on Clustering

- K-means assumes spherical clusters, requires pre-set k , struggles with noise
- DBSCAN handles noise, no k needed, but uses fixed density threshold

Why HDBSCAN?

- Useful when data has non-uniform density or complex shapes
- No need to specify number of clusters
- Creates a hierarchy of density-based clusters (multi-resolution)
- Automatically detects noise/outliers
- Supports soft clustering with membership probabilities
- Robust to varying density $\rightarrow$ ideal for blurry, imprecise data

Mathematics Behind HDBSCAN

- Step 1: Mutual Reachability Distance

$$\text{core}_k(x) = d(x, k^{\text{th}} \text{NN})$$

$$d_{\text{mreach}}(x, y) = \max\{\text{core}_k(x), \text{core}_k(y), d(x, y)\}.$$

- Step 2: Minimal Spanning Tree Construction

$$T^* = \arg \min_{T \text{ spanning } G} \sum_{(x_i, x_j) \in T} d_{\text{mreach}}(x_i, x_j).$$

- Step 3: Hierarchical Clustering via Single Linkage

$$C^{(k)} = C^{(k-1)} \setminus \{C_i, C_j\} \cup \{C_i \cup C_j\}$$

- Step 4: Cluster Hierarchy Extraction

$$C_\lambda = \text{connected components of } \{e_k \mid w_k \leq \lambda\}, \text{ for } \lambda \in [0, \infty).$$

- Step 5: Condensed Tree & Stability Calculation

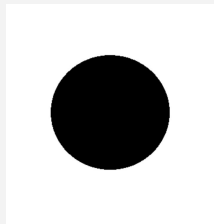
$$\text{stability}(C) = \sum_{x \in C} (\lambda_{\text{death}}(x) - \lambda_{\text{birth}}(x))$$

- Step 6: Optimal Flat Clustering Extraction

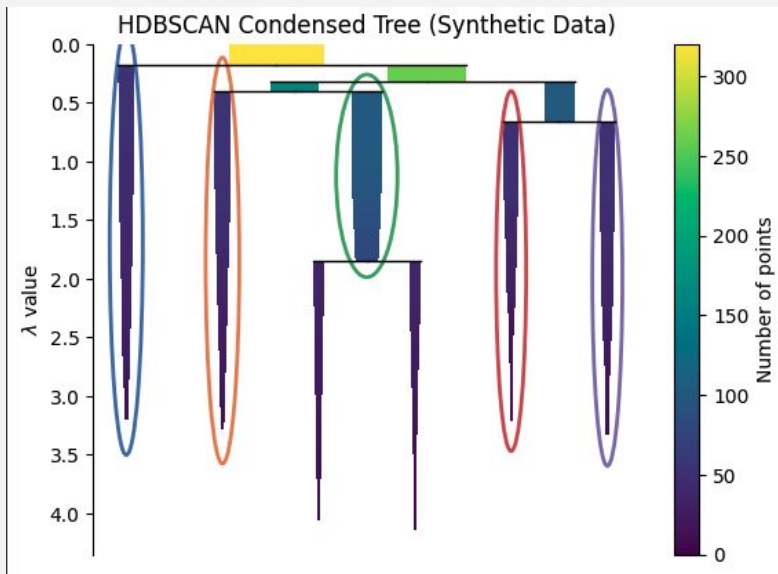
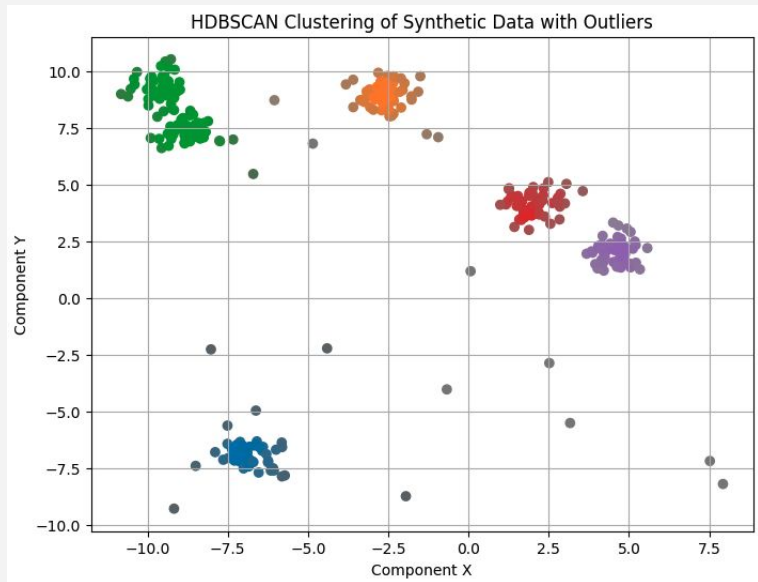
- The most *persistent* clusters, defined as those having high stability across multiple density levels, are selected
- Iteratively traverse the tree and select clusters that maximize stability

Data

- Initial: Randomized matrix
- Test case 1: White background + black circle
- Test case 2: White background + black rectangle
- Simulated dataset: Two blurry semi-circles and one blurry oval



HDBSCAN Simulated Data Example



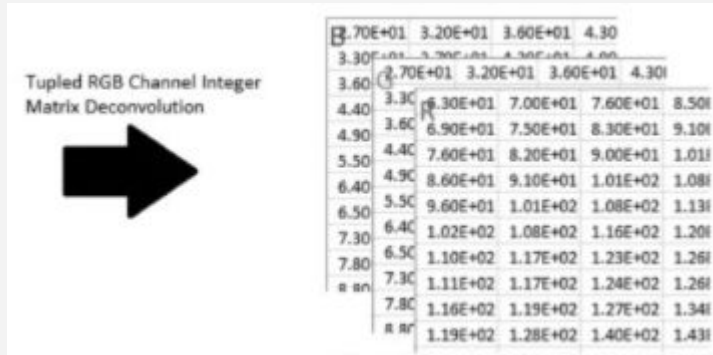
```
# Generate synthetic 2D data with 6 clusters
X, _ = make_blobs(n_samples=300, centers=6, n_features=2, cluster_std=0.5,
                  random_state=42)

# Add some random noise/outliers
outliers = np.random.uniform(low=-10, high=10, size=(20, 2))
X = np.vstack([X, outliers])
```

Image Preprocessing: Tensor Unfolding

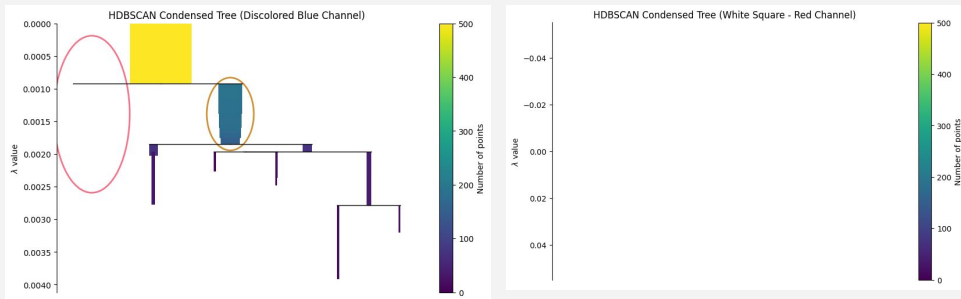
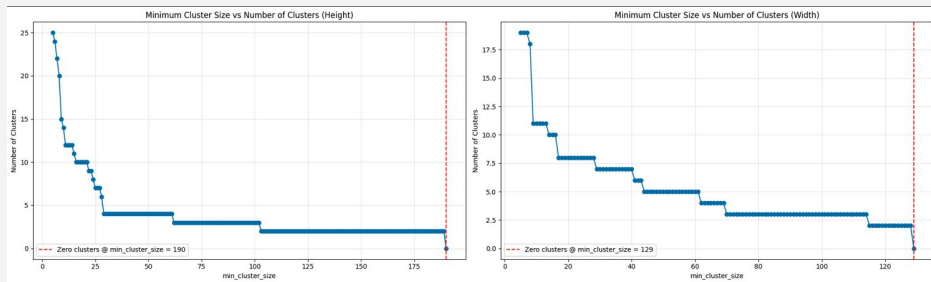
- Every pixel of an image is composed of a tuple of 3 integers (R,G,B)
- Python algorithm used to deconvolute images into 3 matrices, corresponding to the R, G, B channels composing an individual image

Image in .png format



Novel Process and Application of HDBSCAN

- “min_cluster_size” parameter sweeps
- Iteratively test values of parameter until no clusters are detected



Methods

- Test case: Use HDBSCAN to calculate the dimensions of clear images (no blurriness)
 - Black circle on white background: calculate the diameter
 - Black rectangle: calculate height and width
- Use case: Implement HDBSCAN on two blurry semi-circles and one blurry oval
 - Visual 1: HDBSCAN condensed tree with cluster membership information
 - Visual 2: Plot of `min_cluster_size` against number of clusters to test all values of `min_cluster_size` until no clustering



```
FOR each subplot (Height and Width):
    Initialize empty lists to track min_cluster_size and number of clusters
    Set starting min_cluster_size = 5

    WHILE number of clusters > 0:
        Fit HDBSCAN clustering on data (or its transpose)
        Count number of non-noise clusters (labels ≠ -1)
        Store current min_cluster_size and cluster count

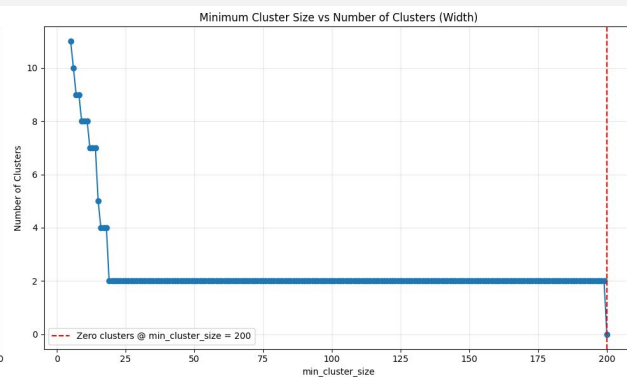
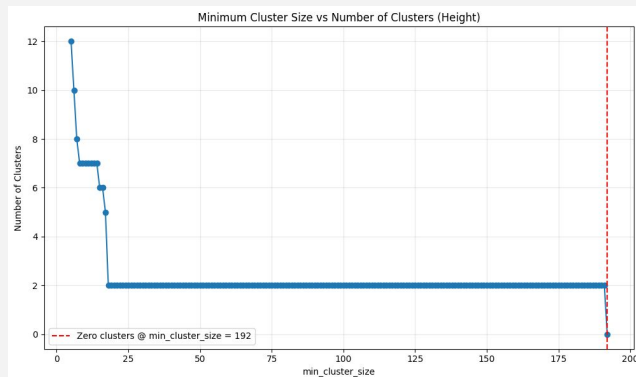
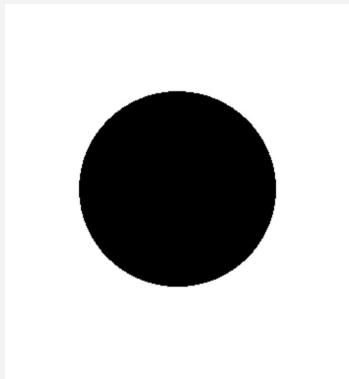
        IF cluster count == 0:
            Break the loop
        ELSE:
            Increment min_cluster_size by 1

    Plot min_cluster_size vs number of clusters
    Mark the point where no clusters are found
```

- Original matrix: Use value of `min_cluster_size` when number of clusters = 0 to determine **height**
- Original matrix *transpose*: Use value of `min_cluster_size` when number of clusters = 0 to determine **width**

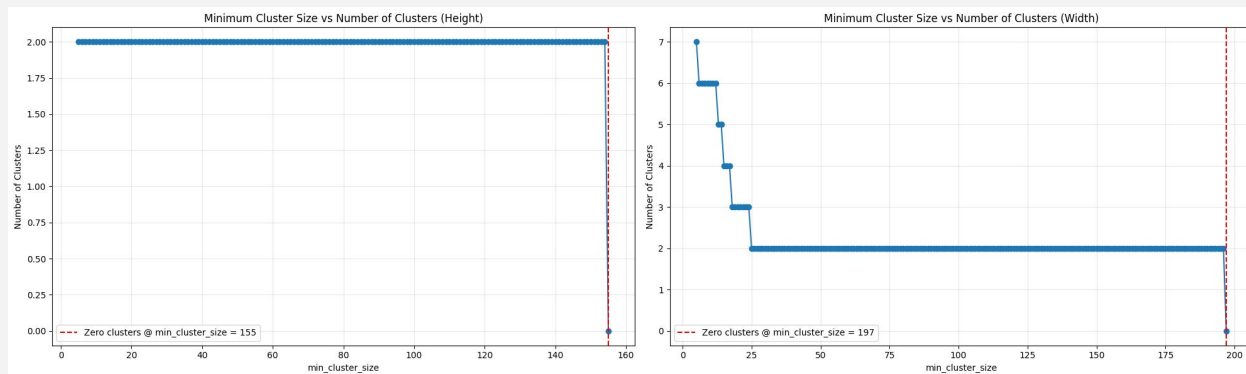
Near Perfect Circle

- True Dimension: 193 (H) X 201 (W)



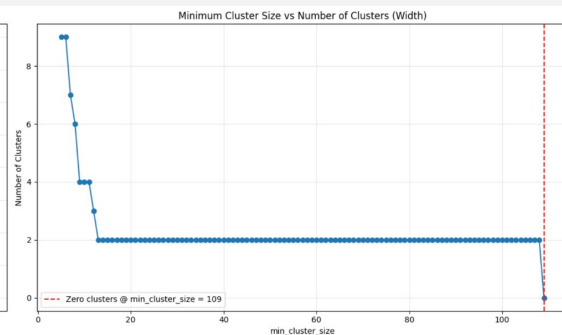
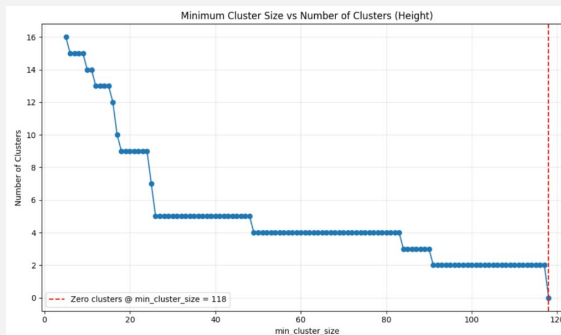
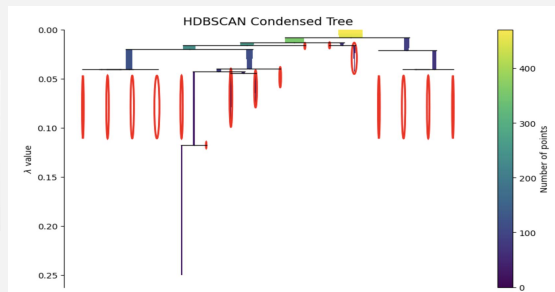
Black Rectangle

- True Dimension: 150 (H) x 300 (W)

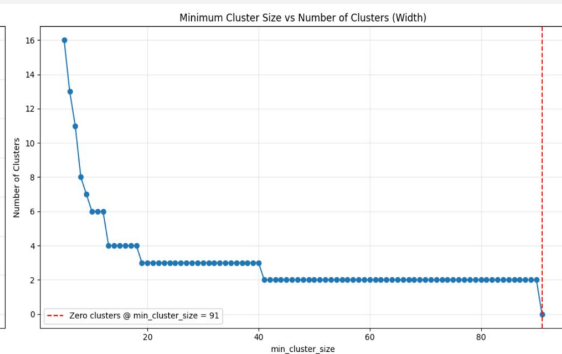
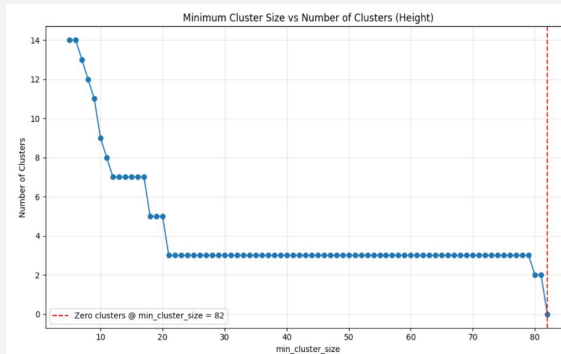
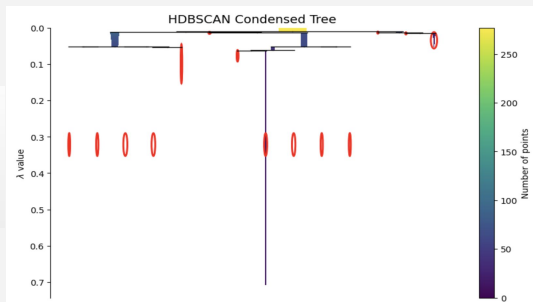


Blurry Semi-Circle Results

Blurry semi-circle 1:



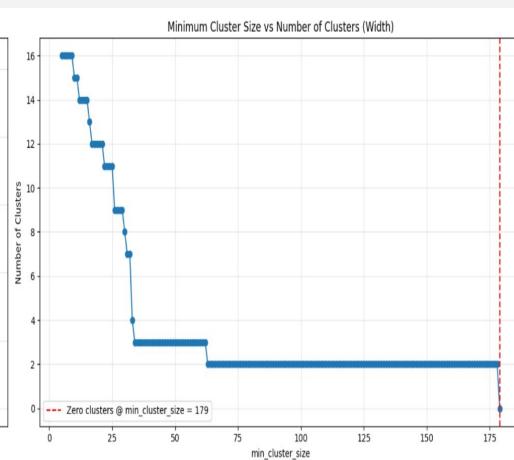
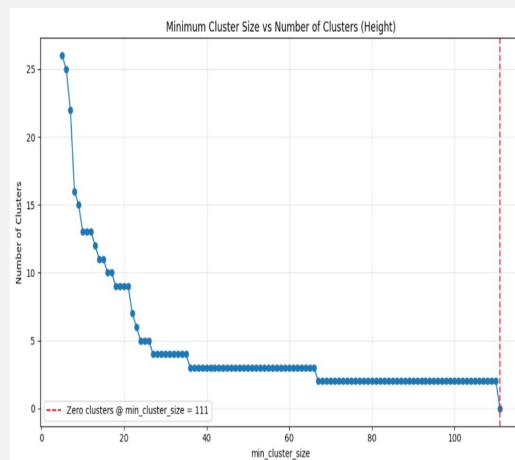
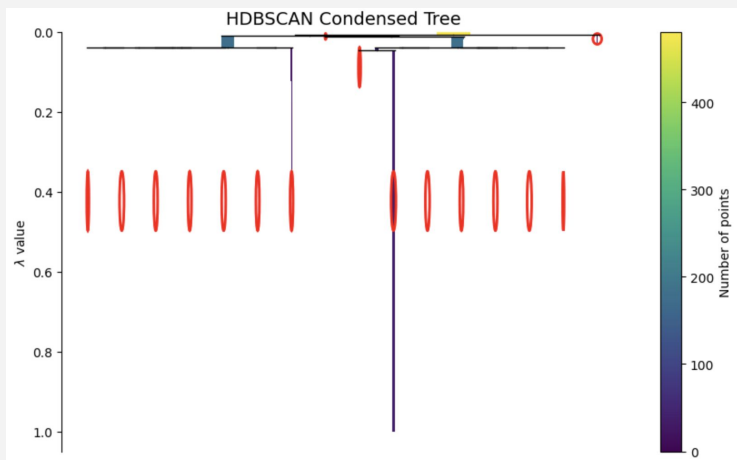
Blurry semi-circle 2:



Blurry Oval Results + Limitations



Blurry Oval:



Limitations:

- Unable to detect the dimensions of the rectangle and Columbia lion after deconvolution; one dimension is correct and the other is incorrect
- Likely due to thin shapes, rough non-continuous edges, or variable-density regions that don't form compact clusters



Thank you!



Literature Review

- An Intuitionistic Fuzzy Clustering Approach for Detection of Abnormal Regions in Mammogram Images
 - <https://pmc.ncbi.nlm.nih.gov/articles/PMC8289970/>
- An Outlier Removal Method Based on PCA-DBSCAN for Blood-SERS Data Analysis
 - <https://pubs.rsc.org/en/content/articlelanding/2024/ay/d3ay02037a>
- Moving Object Detection Based on Clustering and Event-Based Cameras
 - <https://ieeexplore.ieee.org/document/10209469>